Lab-1_Intro2NeuroImagingData_Segmentation

October 10, 2026

1 Anatomical Magnetic Resonance Imaging (MRI) data

This session will guide you through the steps that one needs to follow in order to understand and work with neuroimaging data. We will cover how this data is obtained and what can be seen from it. NeuroImaging data is sensitive, heavy, complex and, most importantly, collaborative. Thus, it needs to be stored in efficient and standardized forms: the NIFTI and BIDS formats. Unfortunately MR scaners do not produce these types of formats and data usually needs to be semiautomatically transformed in order to meet the aforementioned criteria.

You will also learn what softwares and/or custom codes one can use in order to inspect the images once they are in the correct format. In order to know what can be done with this type of data is perhaps more important to be aware at what you are not looking at. Understanding what is the data showing you is a crucial step prior to digging into the specifics of the data structure and what can be done with it.

Eventually, you will be in a position where you can do what you came here to do… use some machine learning or artificial intelligence methods to unravel what the naked eye might have missed. One of the gold-standard of medical imaging is the segmentation of different structures or lesions that can be seen in the brain. Only after doing a manual segmentation can one see the huge advantage of letting stochastic gradient descent do the heavy work.

The notebook is organized as follows 1. From the MR scanner to your laptop: DICOM, NIFTI and BIDS 2. Data and visualization: What do I have? How do I see what I have? 3. Tissue segmentation: Unsupervised vs supervised methods 4. Anatomical MRI acquisitions: What am I looking at? 5. Homework

Just to be on the safe do what you probably already know:

```
conda create -y -n ML4Neuro python=3.11 numpy matplotlib
conda init bash
conda activate ML4Neuro
conda install -c conda-forge jupyterlab
cd /path/to/github/directory
jupyter-lab
```

1.1 1. From the MR scanner to your laptop: DICOM, NIFTI and BIDS.

The *Digital Imaging and Communications in Medicine* (DICOM) is the standard format for storing and transmitting medical imaging data and information. In this section we will take a look at what this format offers as well as its shortcomings. Ultimately, we need to convert from DICOM

to NIFTI and reorganize our data in a way that it's compliant with the BIDS format. This organizational format ensures a correct usage of neuroimaging software increasing the reproducibility and replicability of neuroimaging findings.

[12]:
```
#!pip install pydicom
#!pip install matplotlib
#!pip install numpy
import pydicom as dicom
import glob
import matplotlib.pylab as plt
import numpy as np
```

[13]:
```
# Prepare the directory variables
DICOM_subject = 'FCS-02AMC'
DICOM_acq = '4'
DICOM_dir = f'./DICOM-dir/{DICOM_subject}/{DICOM_acq}/'
slices = sorted(glob.glob(DICOM_dir+'DICOM/*.dcm'))
```

[14]:
```
# Read a single DICOM image
dicom_image = dicom.dcmread(slices[0])
dicom_image
```

```
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-
packages/pydicom/valuerep.py:440: UserWarning: The value length (22) exceeds the
maximum length of 16 allowed for VR SH.
  warn_and_log(msg)
```

[14]:
```
Dataset.file_meta -------------------------------
(0002,0000) File Meta Information Group Length  UL: 198
(0002,0001) File Meta Information Version       OB: b'\x00\x01'
(0002,0002) Media Storage SOP Class UID         UI: MR Image Storage
(0002,0003) Media Storage SOP Instance UID      UI:
1.3.12.2.1107.5.2.32.35177.3.20101116114940227370557 2
(0002,0010) Transfer Syntax UID                 UI: Explicit VR Little Endian
(0002,0012) Implementation Class UID            UI: 1.2.40.0.13.1.1
(0002,0013) Implementation Version Name         SH: 'dcm4che-2.0'
(0002,0016) Source Application Entity Title     AE: 'DCMSEND'
-------------------------------------------------
(0008,0005) Specific Character Set              CS: 'ISO_IR 100'
(0008,0008) Image Type                          CS: ['ORIGINAL', 'PRIMARY', 'M',
'ND', 'NORM']
(0008,0012) Instance Creation Date              DA: '20101116'
(0008,0013) Instance Creation Time              TM: '114942.328000'
(0008,0016) SOP Class UID                       UI: MR Image Storage
(0008,0018) SOP Instance UID                    UI:
1.3.12.2.1107.5.2.32.35177.3.20101116114940227370557 2
(0008,0020) Study Date                          DA: '20101116'
(0008,0021) Series Date                         DA: '20101116'
```

```
(0008,0022) Acquisition Date                    DA: '20101116'
(0008,0023) Content Date                        DA: '20101116'
(0008,0030) Study Time                          TM: '114152.078000'
(0008,0031) Series Time                         TM: '114942.312000'
(0008,0032) Acquisition Time                    TM: '114508.802500'
(0008,0033) Content Time                        TM: '114942.328000'
(0008,0060) Modality                            CS: 'MR'
(0008,0070) Manufacturer                        LO: 'SIEMENS'
(0008,0080) Institution Name                    LO: 'CCIR_00299'
(0008,0081) Institution Address                 ST: 'Scott Ave.4525, St. Louis,
MO, US, 63110'
(0008,0090) Referring Physician's Name          PN: ''
(0008,1010) Station Name                        SH: 'MEDPC'
(0008,1030) Study Description                   LO: 'CCIR_00299'
(0008,103E) Series Description                  LO: 'MPR_1mm_iso'
(0008,1050) Performing Physician's Name         PN: ''
(0008,1070) Operators' Name                     PN: 'Nick Metcalf'
(0008,1090) Manufacturer's Model Name           LO: 'TrioTim'
(0008,1140)  Referenced Image Sequence  3 item(s) ----
   (0008,1150) Referenced SOP Class UID            UI: MR Image Storage
   (0008,1155) Referenced SOP Instance UID         UI:
1.3.12.2.1107.5.2.32.35177.3.20101116114220174724041 56
   ---------
   (0008,1150) Referenced SOP Class UID            UI: MR Image Storage
   (0008,1155) Referenced SOP Instance UID         UI:
1.3.12.2.1107.5.2.32.35177.3.20101116114223553480416 0
   ---------
   (0008,1150) Referenced SOP Class UID            UI: MR Image Storage
   (0008,1155) Referenced SOP Instance UID         UI:
1.3.12.2.1107.5.2.32.35177.3.20101116114217295998041 52
   ---------
(0010,0010) Patient's Name                      PN: 'FCS_002_AMC'
(0010,0020) Patient ID                          LO: 'FCS-02AMC'
(0010,0040) Patient's Sex                       CS: 'M'
(0010,1030) Patient's Weight                    DS: '67.5852717273'
(0010,4000) Patient Comments                    LT: Array of 18 elements
(0012,0063) De-identification Method            LO: 'CNDA common
deidentification'
(0012,0064)  De-identification Method Code Sequence  1 item(s) ----
   (0008,0100) Code Value                          SH: '2295161'
   (0008,0102) Coding Scheme Designator            SH: 'XNAT'
   (0008,0103) Coding Scheme Version               SH: '0.1'
   (0008,0104) Code Meaning                        LO: 'XNAT Edit Script'
   ---------
(0018,0015) Body Part Examined                  CS: 'HEAD'
(0018,0020) Scanning Sequence                   CS: ['GR', 'IR']
(0018,0021) Sequence Variant                    CS: ['SP', 'MP']
```

```
(0018,0022) Scan Options                          CS: 'IR'
(0018,0023) MR Acquisition Type                   CS: '3D'
(0018,0024) Sequence Name                         SH: '*tfl3d1_ns'
(0018,0025) Angio Flag                            CS: 'N'
(0018,0050) Slice Thickness                       DS: '1'
(0018,0080) Repetition Time                       DS: '1950'
(0018,0081) Echo Time                             DS: '2.26'
(0018,0082) Inversion Time                        DS: '900'
(0018,0083) Number of Averages                    DS: '1'
(0018,0084) Imaging Frequency                     DS: '123.255811'
(0018,0085) Imaged Nucleus                        SH: '1H'
(0018,0086) Echo Number(s)                        IS: '1'
(0018,0087) Magnetic Field Strength               DS: '3'
(0018,0089) Number of Phase Encoding Steps        IS: '255'
(0018,0091) Echo Train Length                     IS: '1'
(0018,0093) Percent Sampling                      DS: '100'
(0018,0094) Percent Phase Field of View           DS: '100'
(0018,0095) Pixel Bandwidth                       DS: '200'
(0018,1000) Device Serial Number                  LO: '35177'
(0018,1020) Software Versions                     LO: 'syngo MR B13 4VB13A'
(0018,1030) Protocol Name                         LO: 'MPR_1mm_iso'
(0018,1251) Transmit Coil Name                    SH: 'Body'
(0018,1310) Acquisition Matrix                    US: [0, 256, 256, 0]
(0018,1312) In-plane Phase Encoding Direction     CS: 'ROW'
(0018,1314) Flip Angle                            DS: '9'
(0018,1315) Variable Flip Angle Flag              CS: 'N'
(0018,1316) SAR                                   DS: '0.12808420634112'
(0018,1318) dB/dt                                 DS: '0'
(0018,5100) Patient Position                      CS: 'HFS'
(0019,0010) Private Creator                       LO: 'SIEMENS MR HEADER'
(0019,1008) [CSA Image Header Type]               CS: 'IMAGE NUM 4'
(0019,1009) [CSA Image Header Version ??]         LO: '1.0'
(0019,100B) [SliceMeasurementDuration]            DS: '271932.5'
(0019,100F) [GradientMode]                        SH: 'Normal'
(0019,1011) [FlowCompensation]                    SH: 'No'
(0019,1012) [TablePositionOrigin]                 SL: [0, 0, -1294]
(0019,1013) [ImaAbsTablePosition]                 SL: [0, 0, -1294]
(0019,1014) [ImaRelTablePosition]                 IS: [0, 0, 0]
(0019,1015) [SlicePosition_PCS]                   FD: [-87.45180555,
-174.81262678, 140.63112565]
(0019,1017) [SliceResolution]                     DS: '1'
(0019,1018) [RealDwellTime]                       IS: '9800'
(0020,000D) Study Instance UID                    UI:
1.3.12.2.1107.5.2.32.35177.30000010111520253648400000010
(0020,000E) Series Instance UID                   UI:
1.3.12.2.1107.5.2.32.35177.3.20101116114501483448054880.0.0.0
(0020,0010) Study ID                              SH: '1'
```

```
(0020,0011) Series Number                        IS: '4'
(0020,0012) Acquisition Number                   IS: '1'
(0020,0013) Instance Number                      IS: '1'
(0020,0032) Image Position (Patient)             DS: [-87.451805546338,
-174.81262677656, 140.63112564927]
(0020,0037) Image Orientation (Patient)          DS: [0.0532575707178,
0.99646343931428, 0.06499419413445, -0.0497353660302, 0.06765271089161,
-0.996468516349]
(0020,0052) Frame of Reference UID               UI:
1.3.12.2.1107.5.2.32.35177.1.20101116114152218.0.0.0
(0020,1040) Position Reference Indicator         LO: ''
(0020,1041) Slice Location                       DS: '-85.983480442818'
(0028,0002) Samples per Pixel                    US: 1
(0028,0004) Photometric Interpretation           CS: 'MONOCHROME2'
(0028,0010) Rows                                 US: 256
(0028,0011) Columns                              US: 256
(0028,0030) Pixel Spacing                        DS: [1, 1]
(0028,0100) Bits Allocated                       US: 16
(0028,0101) Bits Stored                          US: 12
(0028,0102) High Bit                             US: 11
(0028,0103) Pixel Representation                 US: 0
(0028,0106) Smallest Image Pixel Value           US: 0
(0028,0107) Largest Image Pixel Value            US: 318
(0028,1050) Window Center                        DS: '48'
(0028,1051) Window Width                         DS: '138'
(0028,1055) Window Center & Width Explanation    LO: 'Algo1'
(0029,0010) Private Creator                      LO: 'SIEMENS CSA HEADER'
(0029,0011) Private Creator                      LO: 'SIEMENS MEDCOM HEADER2'
(0029,1008) [CSA Image Header Type]              CS: 'IMAGE NUM 4'
(0029,1009) [CSA Image Header Version]           LO: '20101116'
(0029,1010) [CSA Image Header Info]              OB: Array of 9212 elements
(0029,1018) [CSA Series Header Type]             CS: 'MR'
(0029,1019) [CSA Series Header Version]          LO: '20101116'
(0029,1020) [CSA Series Header Info]             OB: Array of 48528 elements
(0029,1160) [Series Workflow Status]             LO: 'com'
(0032,1060) Requested Procedure Description      LO: 'head CORBETTA'
(0040,0244) Performed Procedure Step Start Date DA: '20101116'
(0040,0245) Performed Procedure Step Start Time TM: '114152.125000'
(0040,0253) Performed Procedure Step ID          SH: 'MR20101116114152'
(0040,0254) Performed Procedure Step Descriptio LO: 'head^CORBETTA'
(0051,0010) Private Creator                      LO: 'SIEMENS MR HEADER'
(0051,1008) [CSA Image Header Type]              CS: 'IMAGE NUM 4'
(0051,1009) [CSA Image Header Version ??]        LO: '1.0'
(0051,100A) [Unknown]                            SH: 'TA 04:31'
(0051,100B) [AcquisitionMatrixText]              SH: '256*256s'
(0051,100C) [Unknown]                            SH: 'FoV 256*256'
(0051,100D) [Unknown]                            SH: 'SP R86.0'
```

```
(0051,100E) [Unknown]                                SH: 'Sag>Tra(3.1)>Cor(2.9)'
(0051,100F) [CoilString]                             LO: 't:HEA;HEP'
(0051,1011) [PATModeText]                            LO: 'p2'
(0051,1012) [Unknown]                                SH: 'TP 0'
(0051,1013) [PositivePCSDirections]                  SH: '+LPH'
(0051,1016) [Unknown]                                LO: 'p2 M/ND/NORM'
(0051,1017) [Unknown]                                SH: 'SL 1.0'
(0051,1019) [Unknown]                                LO: 'A1/IR'
(7FE0,0010) Pixel Data                               OW: Array of 131072 elements
```

This is a lot of information, which may or may not be useful depending on the experiment or on the type of paper your are trying to write. Given that `dicom_image` is a python object, we can access whatever fields we wish. Several acquisition parameters are usually reported in the studies and all of them can easily be accessed through this object. It is common to have the same scanning protocols for all the subjects in the dataset, but sometimes mistakes occur. With this type of software it should be easy to automatize searching schemes in order to stablish whether the protocol was changed during a study.

[15]:
```
# What was the Repetition Time of the protocol?
TR = dicom_image.RepetitionTime
print(f"Repetition Time = {TR} (s)")
# What was the Echo Time of the protocol?
ET = dicom_image.EchoTime
print(f"Echo Time = {ET} (s)")
# What was the Type of acquisition?
ACQ_type = dicom_image.SeriesDescription
print(f"Acquisition type: {ACQ_type}")
```

```
Repetition Time = 1950 (s)
Echo Time = 2.26 (s)
Acquisition type: MPR_1mm_iso
```

However, can we acess the data? If so, what is the data exactly?

[16]:
```
data = dicom_image.pixel_array
print(f"The shape of the data is: {data.shape}")
print(f"The type of variable is: {type(data)}")
print(data)
```

```
The shape of the data is: (256, 256)
The type of variable is: <class 'numpy.ndarray'>
[[ 0  0  0 …  0  0  0]
 [ 0  2  2 …  2  1  0]
 [ 0  1  1 …  1  4  1]
 …
 [ 0  0  5 … 13 29 13]
 [ 0  5  6 … 10  7 21]
 [ 0  7  4 …  9 21  3]]
```

Question: What are the main differences between acquisition '16' and '4' of the subject with ID

'FCS-02AMC'?

Answer:

Acquisition 4 is a T1-weighted scan. It is one sharp 3D image (1 mm voxels) that shows brain *anatomy*: gray matter, white matter and fluid each have a different brightness. It's used for segmentation and registration.

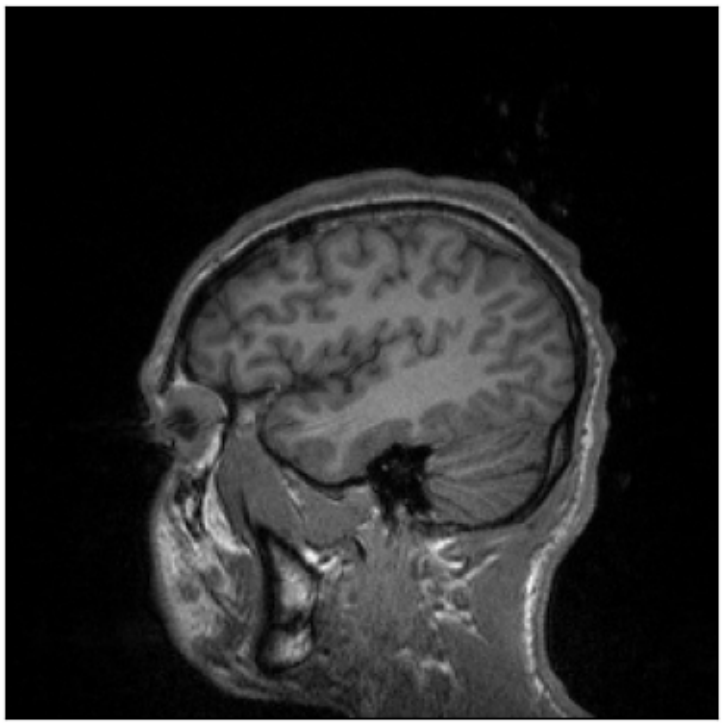
Acquisition 16 is a diffusion (DTI) scan. It is 64 lower-resolution 3D volumes (2 mm voxels) that measure *how water moves* in different directions. Water moves along nerve fibres, so this scan is used to map brain *connections*. It needs `.bval`/`.bvec` files (the strength and direction of each measurement), and it uses much longer TR/TE (9.2 s / 92 ms vs 1.95 s / 2.26 ms), so its images are noisier.

It is clear that the DICOM format stores a lot of information including the data itself. Yet, what is the data? How can you see what you have stored? And how compact and easy to use is this format when it comes to numerical analyses of the scans?

[17]:

```
num_to_plot = 1

fig, axes = plt.subplots(num_to_plot, num_to_plot)
if num_to_plot > 1:
  axes = axes.flatten()
  for i, ax in enumerate(axes):
    ax.imshow(
        dicom.dcmread(slices[i]).pixel_array, cmap='gray'
    )
    ax.set_xticks([]), ax.set_yticks([])
else:
  axes.imshow(
      dicom.dcmread(slices[35]).pixel_array, cmap='gray'
  )
  axes.set_xticks([]), axes.set_yticks([])
plt.show()
```



Ares these images the same? Recall that the brain (or any other organ) is a 3D volume, and current MR scanners take 2D pictures hence they have to move along the head volume capturing *slices* of the subject. The DICOM format stores all these slices together with the corresponding information for each slice (e.g., Window Center). Each slice is stored separately in the acquisition folder (e.g., 16) with a very strange and almost undesciferable name. Furthermore, we can only visualize a single viewing plane that depends on the acquisition parameters. In the former example we could plot the sagital plane straight from the data. Can we do better? Yes!

The *Neuroimaging Informatics Technology Initiative* (NIfTI) format is commonly used brain MR data and it allows to store all the desired information available in the DICOM directory into a single file. Importantly, there is a standard file-naming code that should be taken into consideration: **the FILE-NAME should not be left unattended!** For every subject, a whole DICOM session will be condensed into:

- a FILE-NAME.nii file containing the data
- a FILE-NAME.json file containing the aquisition parameters
- a FILE-NAME.bvec file containing gradient directions
- a FILE-NAME.bval file containing the gradient strengths

Importantly, the two last files will only appear in the case of difussion MRI acquisitions. For now let's ignore their meaning since we will focus on them in the next laboratory session.

To move from DICOM to NIFTI formats one could build a custom converter. The basics would require a stacking of all slices along a 3rd dimension ensuring that each slice is positioned correctly along the depth axis. A proper json file should also be manually written with the information extracted from the DICOM heather. Evenmore, in the case of diffusion data, the gradient strength

and orientation would also need to be decoded from the DICOM heather. Yet, as you can probably imagine, there is a better and faster way to do all this. For now, let's convert the previous DICOM session to NIFTI and see what it looks like.

[ ]:

[ ]:

[18]:
```
!pip install dicom2nifti
import dicom2nifti
```

```
Requirement already satisfied: dicom2nifti in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (2.6.2)
Requirement already satisfied: nibabel in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from dicom2nifti)
(5.4.2)
Requirement already satisfied: numpy in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from dicom2nifti)
(2.4.6)
Requirement already satisfied: scipy in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from dicom2nifti)
(1.17.1)
Requirement already satisfied: pydicom>=3.0.0 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from dicom2nifti)
(3.0.2)
Requirement already satisfied: python-gdcm in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from dicom2nifti)
(3.2.6)
Requirement already satisfied: importlib-resources>=5.12 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
nibabel->dicom2nifti) (7.1.0)
Requirement already satisfied: packaging>=20 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
nibabel->dicom2nifti) (26.3)
Requirement already satisfied: typing-extensions>=4.6 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
nibabel->dicom2nifti) (4.16.0)
```

[19]:
```
import os
```

[20]:
```
T1_DICOM = f'./DICOM-dir/{DICOM_subject}/4/'
DTI_DICOM = f'./DICOM-dir/{DICOM_subject}/16/'

NIFTI_dir = f'./subject_{DICOM_subject}_NIFTI_dir/pydicom/'
if not os.path.exists(NIFTI_dir):
  os.makedirs(NIFTI_dir)

# Apply conversion
```

```
dicom2nifti.convert_directory(T1_DICOM, NIFTI_dir, compression=True)
dicom2nifti.convert_directory(DTI_DICOM, NIFTI_dir, compression=True)
```

Where are the `.json` files? Have we lied to you before? No!

Exercise *Find a way to obtain the missing `.json` with the `pydicom` library. The documentation can be found here.

[ ]:
```
# pydicom.to_json_dict() exports DICOM JSON, not a NIfTI/BIDS sidecar.
# The linked documentation is for dicom2nifti, a different library.
# To obtain the sidecars intended here, use dcm2niix (introduced below).
# Convert images and metadata together so that each JSON matches its NIfTI.
%pip install dcm2niix

from pathlib import Path
import json
import shutil
import subprocess
import sys

# Prefer the executable installed in this notebook's Python environment.
dcm2niix = Path(sys.executable).parent / "dcm2niix"
if not dcm2niix.is_file():
    executable = shutil.which("dcm2niix")
    if executable is None:
        raise FileNotFoundError("dcm2niix installation failed; check the pip␣
 ↪output above.")
    dcm2niix = Path(executable)

output_dir = Path(f"subject_{DICOM_subject}_NIFTI_dir/dcm2niix-sidecars")
output_dir.mkdir(parents=True, exist_ok=True)

for acquisition in ["4", "16"]:
    input_dir = Path("DICOM-dir") / DICOM_subject / acquisition
    if not list(input_dir.rglob("*.dcm")):
        raise FileNotFoundError(f"No DICOM files found in {input_dir}")

    subprocess.run([
        str(dcm2niix),
        "-b", "y",        # Write BIDS-style JSON sidecars.
        "-ba", "y",       # Omit identifying information from the sidecars.
        "-z", "y",        # Compress NIfTI images to .nii.gz.
        "-f", f"{DICOM_subject}_%s_%p",  # Subject, series number, protocol.
        "-o", str(output_dir),
        str(input_dir),
    ], check=True)

for json_path in sorted(output_dir.glob("*.json")):
```

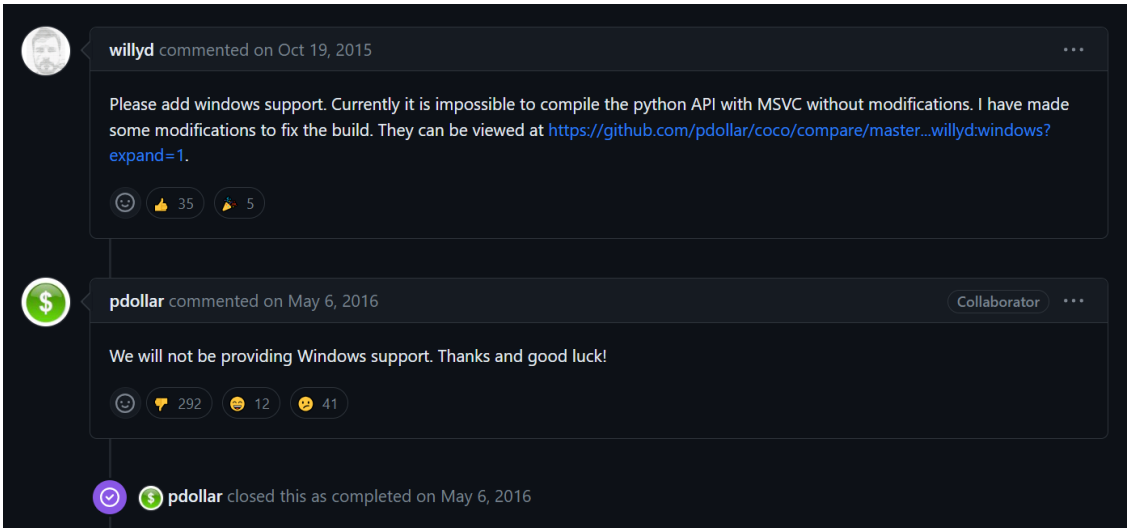
```
    metadata = json.loads(json_path.read_text())
    print(json_path.name)
    print("  TR (s):", metadata.get("RepetitionTime"))
    print("  TE (s):", metadata.get("EchoTime"))

# These sidecars accompany the new NIfTI files in dcm2niix-sidecars/.
# TR and TE here are in seconds (the original DICOM values are in ms).
# BIDS folder organization and naming are a separate exercise below.
```

```
Collecting dcm2niix
  Downloading dcm2niix-1.0.20260724-cp38-abi3-macosx_11_0_arm64.whl.metadata (34
kB)
Downloading dcm2niix-1.0.20260724-cp38-abi3-macosx_11_0_arm64.whl (941 kB)
                        941.6/941.6 kB
7.9 MB/s  0:00:00
Installing collected packages: dcm2niix
Successfully installed dcm2niix-1.0.20260724
Note: you may need to restart the kernel to use updated packages.
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 176 DICOM file(s)
Convert 176 DICOM as subject_FCS-02AMC_NIFTI_dir/dcm2niix-
sidecars/FCS-02AMC_4_MPR_1mm_iso (256x256x176x1)
Conversion required 0.814258 seconds (0.705377 for core code).
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 64 DICOM file(s)
Convert 64 DICOM as subject_FCS-02AMC_NIFTI_dir/dcm2niix-
sidecars/FCS-02AMC_16_dti_63dir_2mm_iso_1of2 (128x128x64x64)
Conversion required 2.664716 seconds (2.536051 for core code).
FCS-02AMC_16_dti_63dir_2mm_iso_1of2.json
  TR (s): 9.2
  TE (s): 0.092
FCS-02AMC_4_MPR_1mm_iso.json
  TR (s): 1.95
  TE (s): 0.00226
```

Have you found a way to do so? If you did, please show it to us. If you didn't succeed do not worry too much because we don't know how to do it either.

Besides, you may have noticed that the FILE-NAME is a strange combination of letters and numbers. In fact, if you try to convert other dicom series you will notice how the name is identical. While this indicates that the protocol for every subject was kept the same, it can be problematic when it comes to identifying subjects and working with the whole dataset.The proper way to do so is to work with LINUX/MACOS, Windows is not really an option:



Once you have proper operating system you can proceed. Alternatively there are virtual machines or docker containers with most of the necessary neuroimaging software which might work properly on Windows: Neurodesk for example. Anyhow, the best way to convert between formats is using dmc2niix.

[ ]:
```
!pip install dcm2niix # This will install the software package but will only be␣
↪available within the current conda environment
# sudo apt install dcm2niix will install it system wide in Linux (requires sudo␣
↪password)
```

[ ]:
```
%%bash

DICOM_dir="./DICOM-dir/FCS-02AMC/"
NIFTI_dir_dcm2niix="./subject_FCS-02AMC_NIFTI_dir/dcm2niix/"
mkdir $NIFTI_dir_dcm2niix

dcm2niix -o $NIFTI_dir_dcm2niix $DICOM_dir
```

As you can see the `.json` files are automatically generated and the conversion is much faster: we converted a whole subject in less than 0.5s. It is recommended to zip the .nii files using the `-z` flag. Furthermore, you have more control on the output name if you use the `-f` flag. You can customize your name in any way you prefer so that you can easily identify the files you converted.

[ ]:
```
%%bash

DICOM_dir="./DICOM-dir/FCS-02AMC/"
NIFTI_dir_dcm2niix_naming="./subject_FCS-02AMC_NIFTI_dir/dcm2niix-naming/"
mkdir $NIFTI_dir_dcm2niix_naming

dcm2niix -o $NIFTI_dir_dcm2niix_naming -f %n--%p--%t $DICOM_dir
```

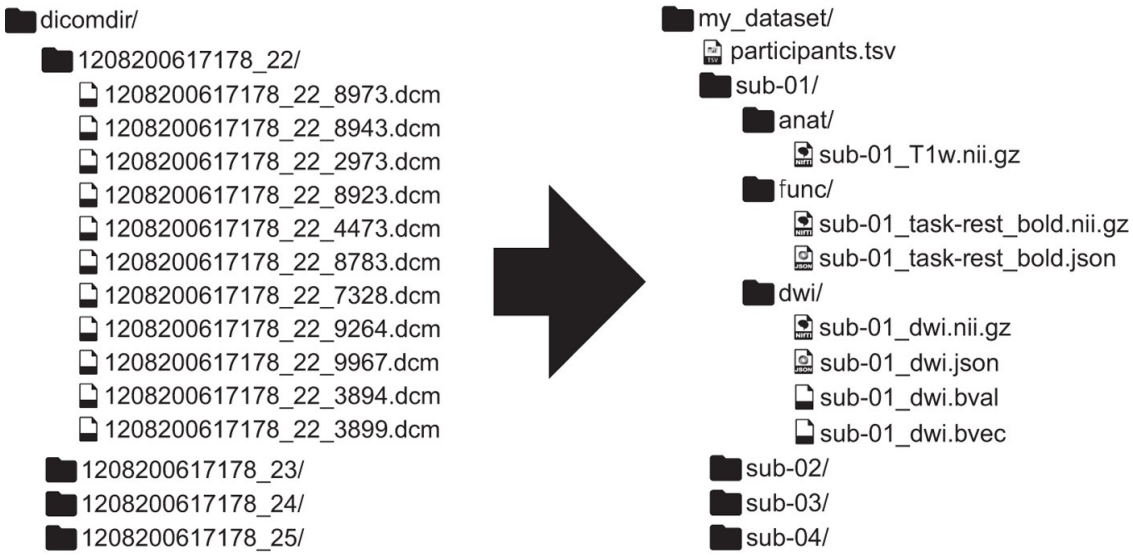
Don't worry you don't have to remember everything we say. Doing research involves a lot of google search queries, so one basically becomes very profficient and precise about the words used in the

search bar. Here are a couple of useful links that might come in handy: BIDS and dcm2niix.

Exercise: *Try to convert the whole DICOM directory to NIFTI files and reorganize them in a way that is BIDS compliant. You can reorganize them manually or automatically using regular expressions and the* `os` *library. To be sure the final dataset agrees with the minimum BIDS requirements you can use any of the two following options:*

1. The python interface (although it is not the best): `pip install bids-validator`
2. The web interface bids-validator

Be sure to include all the necessary information when converting your dicom files so that you can identify them properly.



[26]:
```
from pathlib import Path
import csv
import json
import shutil
import subprocess
import sys
import tempfile
import re
import pydicom

# Run the dcm2niix installation cell above first.
source_dir = Path('DICOM-dir')
BIDS_dir = Path('BIDS-converted')  # Keep the supplied BIDS-dir unchanged.
BIDS_dir.mkdir(exist_ok=True)
dcm2niix = Path(sys.executable).parent / 'dcm2niix'
if not dcm2niix.is_file():
    executable = shutil.which('dcm2niix')
    if executable is None:
```

```
        raise FileNotFoundError('Install dcm2niix using the earlier notebook␣
 ↪cell.')
    dcm2niix = Path(executable)

# Preserve source-folder identity; do not infer repeat visits from folder names.
subject_dirs = sorted(p for p in source_dir.iterdir() if p.is_dir())
subject_ids = {p.name: re.sub(r'[^A-Za-z0-9]', '', p.name).upper()
               for p in subject_dirs}
if len(set(subject_ids.values())) != len(subject_ids):
    raise ValueError('Source names produce duplicate BIDS subject labels.')

manifest = []
for subject_dir in subject_dirs:
    subject = 'sub-' + subject_ids[subject_dir.name]
    for series_dir in sorted(p for p in subject_dir.iterdir() if p.is_dir()):
        dicoms = sorted(series_dir.rglob('*.dcm'))
        if not dicoms:
            continue
        header = pydicom.dcmread(dicoms[0], stop_before_pixels=True)
        protocol = str(header.SeriesDescription)
        if protocol == 'MPR_1mm_iso':
            datatype, suffix = 'anat', 'T1w'
        elif protocol.startswith('dti_'):
            datatype, suffix = 'dwi', 'dwi'
        else:
            raise ValueError(f'Please classify this protocol: {protocol}')

        # Retain the source series number as an acquisition label.
        basename = f'{subject}_acq-series{int(header.SeriesNumber)}_{suffix}'
        destination = BIDS_dir / subject / datatype
        destination.mkdir(parents=True, exist_ok=True)
        with tempfile.TemporaryDirectory() as temporary:
            result = subprocess.run([
                str(dcm2niix), '-b', 'y', '-ba', 'y', '-z', 'y',
                '-f', basename, '-o', temporary, str(series_dir),
            ], check=True, capture_output=True, text=True)
            # Keep converter warnings available for inspection.
            print(result.stdout)
            if result.stderr:
                print(result.stderr)
            images = list(Path(temporary).glob('*.nii.gz'))
            if len(images) != 1 or images[0].name != basename + '.nii.gz':
                raise ValueError(f'Unexpected outputs for {series_dir}:␣
 ↪{images}')
            extensions = ['.nii.gz', '.json']
            if datatype == 'dwi':
                extensions += ['.bval', '.bvec']
```

```
            for extension in extensions:
                file = Path(temporary) / (basename + extension)
                if not file.is_file():
                    raise FileNotFoundError(file)
            for extension in extensions:
                shutil.copy2(Path(temporary) / (basename + extension),␣
 ↪destination)

        manifest.append({
            'source_directory': series_dir.as_posix(),
            'series_uid': str(header.SeriesInstanceUID),
            'protocol': protocol,
            'dicom_count': len(dicoms),
            'bids_image': (destination / (basename + '.nii.gz')).
 ↪relative_to(BIDS_dir).as_posix(),
        })
        print(f'{series_dir} -> {destination / (basename + ".nii.gz")}')

if not manifest:
    raise ValueError('No DICOM series were found.')

# Dataset-level metadata; do not invent demographic information.
(BIDS_dir / 'dataset_description.json').write_text(json.dumps({
    'Name': 'ML4Neuro Lab 1 DICOM conversion',
    'BIDSVersion': '1.11.2',
    'DatasetType': 'raw',
}, indent=2) + '\n', encoding='utf-8')
(BIDS_dir / 'participants.tsv').write_text(
    'participant_id\n' + ''.join(f'sub-{label}\n' for label in subject_ids.
 ↪values()),
    encoding='utf-8',
)
(BIDS_dir / 'README').write_text(
    'Lab 1: all supplied DICOM series converted with dcm2niix.\n'
    'anat/T1w: structural MRI; dwi: diffusion MRI with bval and bvec files.\n'
    'acq-seriesN preserves the original DICOM series number.\n'
    'Subject labels preserve source folder names (uppercase, alphanumeric).\n'
    'Each source folder is treated as a separate subject identifier for this␣
 ↪exercise.\n'
    'Whether similarly named folders represent repeat visits is not established;
 ↪\n'
    'confirm participant identity before using this dataset for group analysis.
 ↪\n'
    'No demographic data were invented. Source folder mapping:\n' +
    ''.join(f'{name} -> sub-{label}\n' for name, label in subject_ids.items()),
    encoding='utf-8',
```

```
)
# Store conversion provenance outside the raw BIDS dataset.
with Path('BIDS-converted_manifest.tsv').open('w', newline='',␣
 ↪encoding='utf-8') as file:
    writer = csv.DictWriter(file, fieldnames=list(manifest[0]), delimiter='\t')
    writer.writeheader()
    writer.writerows(manifest)
print(f'Converted {len(manifest)} series from {len(subject_dirs)} source␣
 ↪folders into {BIDS_dir}.')
```

```
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 64 DICOM file(s)
Convert 64 DICOM as
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/tmpnqzne2kr/sub-FCS02AMC_acq-
series16_dwi (128x128x64x64)
Conversion required 2.713502 seconds (2.586179 for core code).

DICOM-dir/FCS-02AMC/16 -> BIDS-converted/sub-FCS02AMC/dwi/sub-FCS02AMC_acq-
series16_dwi.nii.gz
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 176 DICOM file(s)
Convert 176 DICOM as
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/tmpj50xa9fq/sub-FCS02AMC_acq-
series4_T1w (256x256x176x1)
Conversion required 0.806974 seconds (0.721403 for core code).

DICOM-dir/FCS-02AMC/4 -> BIDS-converted/sub-FCS02AMC/anat/sub-FCS02AMC_acq-
series4_T1w.nii.gz
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 176 DICOM file(s)
Convert 176 DICOM as
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/tmpwup2qapi/sub-FCS01ACM_acq-
series4_T1w (256x256x176x1)
Conversion required 0.713383 seconds (0.629282 for core code).

DICOM-dir/fcs-01acm/4 -> BIDS-converted/sub-FCS01ACM/anat/sub-FCS01ACM_acq-
series4_T1w.nii.gz
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 64 DICOM file(s)
Convert 64 DICOM as
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/tmpl2x1f7ne/sub-FCS06AMC2_acq-
series14_dwi (128x128x64x64)
Conversion required 2.449167 seconds (2.349493 for core code).
```

```
DICOM-dir/fcs-06amc2/14 -> BIDS-converted/sub-FCS06AMC2/dwi/sub-FCS06AMC2_acq-
series14_dwi.nii.gz
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 176 DICOM file(s)
Convert 176 DICOM as
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/tmpjoxvmu8j/sub-FCS06AMC2_acq-
series4_T1w (256x256x176x1)
Conversion required 0.756776 seconds (0.666733 for core code).

DICOM-dir/fcs-06amc2/4 -> BIDS-converted/sub-FCS06AMC2/anat/sub-FCS06AMC2_acq-
series4_T1w.nii.gz
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 64 DICOM file(s)
Convert 64 DICOM as
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/tmph9jqithm/sub-FCS07ACM_acq-
series15_dwi (128x128x64x64)
Conversion required 2.421576 seconds (2.321549 for core code).

DICOM-dir/fcs-07acm/15 -> BIDS-converted/sub-FCS07ACM/dwi/sub-FCS07ACM_acq-
series15_dwi.nii.gz
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 176 DICOM file(s)
Convert 176 DICOM as
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/tmpcon2ncxv/sub-FCS07ACM_acq-
series4_T1w (256x256x176x1)
Conversion required 0.640592 seconds (0.634959 for core code).

DICOM-dir/fcs-07acm/4 -> BIDS-converted/sub-FCS07ACM/anat/sub-FCS07ACM_acq-
series4_T1w.nii.gz
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 64 DICOM file(s)
Convert 64 DICOM as
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/tmp23iheiuq/sub-FCS07AMC2_acq-
series16_dwi (128x128x64x64)
Conversion required 2.484438 seconds (2.412351 for core code).

DICOM-dir/fcs-07amc2/16 -> BIDS-converted/sub-FCS07AMC2/dwi/sub-FCS07AMC2_acq-
series16_dwi.nii.gz
Chris Rorden's dcm2niiX version v1.0.20260724  (JP2:OpenJPEG) (JP-LS:CharLS)
Clang21.0.0 ARM (64-bit MacOS)
Found 176 DICOM file(s)
Convert 176 DICOM as
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/tmpp8nt6wd6/sub-FCS07AMC2_acq-
series4_T1w (256x256x176x1)
```

```
Conversion required 0.688008 seconds (0.677128 for core code).

DICOM-dir/fcs-07amc2/4 -> BIDS-converted/sub-FCS07AMC2/anat/sub-FCS07AMC2_acq-
series4_T1w.nii.gz
Converted 9 series from 5 source folders into BIDS-converted.
```

[27]:
```
%pip install bids-validator

from bids_validator import BIDSValidator
import nibabel as nib
import numpy as np

# This Python API checks file paths, not full dataset compliance.
validator = BIDSValidator()
files = sorted(p for p in BIDS_dir.rglob('*') if p.is_file())
invalid = [p.relative_to(BIDS_dir).as_posix() for p in files
           if not validator.is_bids('/' + p.relative_to(BIDS_dir).as_posix())]
assert not invalid, f'Invalid BIDS paths: {invalid}'

# Also check image readability, metadata, and diffusion gradient dimensions.
images = sorted(BIDS_dir.rglob('*.nii.gz'))
for image_path in images:
    image = nib.load(image_path)
    basename = image_path.name.removesuffix('.nii.gz')
    metadata = json.loads(image_path.with_name(basename + '.json').read_text())
    assert metadata['RepetitionTime'] > 0
    assert metadata['EchoTime'] > 0
    if basename.endswith('_dwi'):
        assert len(image.shape) == 4, image_path
        bvals = np.atleast_1d(np.loadtxt(image_path.with_name(basename + '.
 ↪bval')))
        bvecs = np.loadtxt(image_path.with_name(basename + '.bvec'))
        assert bvals.shape == (image.shape[3],), image_path
        assert bvecs.shape == (3, image.shape[3]), image_path
        assert np.isfinite(bvals).all() and np.isfinite(bvecs).all()
    print(image_path.relative_to(BIDS_dir), image.shape)

print(f'Passed: {len(files)} BIDS paths; {len(images)} images and their␣
 ↪sidecars.')
print('Full dataset validation: open https://bids-standard.github.io/
 ↪bids-validator/')
print(f'and select the {BIDS_dir} folder. Path checks alone do not certify BIDS␣
 ↪compliance.')
```

```
Collecting bids-validator
  Using cached bids_validator-1.14.7.post0-py3-none-any.whl.metadata (39 kB)
Collecting bidsschematools>=0.10 (from bids-validator)
  Using cached bidsschematools-2.0.0-py3-none-any.whl.metadata (2.9 kB)
```

```
Collecting acres>=0.5 (from bidsschematools>=0.10->bids-validator)
  Using cached acres-0.5.0-py3-none-any.whl.metadata (6.2 kB)
Collecting click>=8.1 (from bidsschematools>=0.10->bids-validator)
  Using cached click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: pyyaml>=6.0 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
bidsschematools>=0.10->bids-validator) (6.0.3)
Using cached bids_validator-1.14.7.post0-py3-none-any.whl (23 kB)
Using cached bidsschematools-2.0.0-py3-none-any.whl (195 kB)
Using cached acres-0.5.0-py3-none-any.whl (12 kB)
Using cached click-8.5.0-py3-none-any.whl (125 kB)
Installing collected packages: click, acres, bidsschematools, bids-validator
                         4/4 [bids-
validator]
Successfully installed acres-0.5.0 bids-validator-1.14.7.post0
bidsschematools-2.0.0 click-8.5.0
Note: you may need to restart the kernel to use updated packages.
sub-FCS01ACM/anat/sub-FCS01ACM_acq-series4_T1w.nii.gz (176, 256, 256)
sub-FCS02AMC/anat/sub-FCS02AMC_acq-series4_T1w.nii.gz (176, 256, 256)
sub-FCS02AMC/dwi/sub-FCS02AMC_acq-series16_dwi.nii.gz (128, 128, 64, 64)
sub-FCS06AMC2/anat/sub-FCS06AMC2_acq-series4_T1w.nii.gz (176, 256, 256)
sub-FCS06AMC2/dwi/sub-FCS06AMC2_acq-series14_dwi.nii.gz (128, 128, 64, 64)
sub-FCS07ACM/anat/sub-FCS07ACM_acq-series4_T1w.nii.gz (176, 256, 256)
sub-FCS07ACM/dwi/sub-FCS07ACM_acq-series15_dwi.nii.gz (128, 128, 64, 64)
sub-FCS07AMC2/anat/sub-FCS07AMC2_acq-series4_T1w.nii.gz (176, 256, 256)
sub-FCS07AMC2/dwi/sub-FCS07AMC2_acq-series16_dwi.nii.gz (128, 128, 64, 64)
Passed: 29 BIDS paths; 9 images and their sidecars.
Full dataset validation: open https://bids-standard.github.io/bids-validator/
and select the BIDS-converted folder. Path checks alone do not certify BIDS
compliance.
```

Sometimes, not all the data is available or useful to us. However, some files are required in order to have a more transparent and ready-to-use dataset. Remember, people are busy so you need to make things friendly for the user. Transparency and replicability are also encouraged even jeopardizing your own publication rates. For your own personal reasons or because the experiment demands it, you can also add a .bidsignore file pointing to files or directories that could be useful. The rationale is the same as the typical .gitignore file. For example, in lesion studies, it might be useful to have a directory with all the lesion masks even if its existence is not contemplated in the BIDS formatting.

Question: What files are you missing? What information should these files have?

Optional exercise: *Add the necessary files (invent data if necessary) so that your directory complies with the validator's demands.*

Answer: A plain dcm2niix conversion only gives images and JSON sidecars. BIDS also needs dataset-level files: **`dataset_description.json`** (required: dataset name and BIDS version; also authors and license), **`participants.tsv`** (one row per subject with ID and, ideally, age, sex and group) and a **`README`** describing the dataset. My conversion script creates these. The official validator then reports no errors, only warnings: no authors listed, sub-FCS01ACM has no DWI scan (it isn't in the DICOM data), and the DWI parameters differ slightly between subjects.

To understand the advantages that the NIFTI and BIDS formats offer it is enough to install and import a single python library. NiBabel The BIDS-dir folder has already some subjects with the correct organization. Feel free to take a look or use the one you created in the last step.

[28]:
```
!pip install nibabel
import nibabel as nib
```

```
Requirement already satisfied: nibabel in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (5.4.2)
Requirement already satisfied: importlib-resources>=5.12 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nibabel) (7.1.0)
Requirement already satisfied: numpy>=1.25 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nibabel) (2.4.6)
Requirement already satisfied: packaging>=20 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nibabel) (26.3)
Requirement already satisfied: typing-extensions>=4.6 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nibabel)
(4.16.0)
```

[43]:
```
nifti = "./BIDS-converted/sub-FCS01ACM/anat/sub-FCS01ACM_acq-series4_T1w.nii.gz"
img1 = nib.load(nifti)
header1, affine1, data1 = img1.header, img1.affine, img1.get_fdata()
```

Nibabel loads all the information present in the NIFTI file onto a python object with three attributes:

1. The **nifti header** contains all the information necessary to interpret the data. It can be used to quickly inspect the number of dimensions, the voxel size, the orientation of the image, … As opposed to the PyDicom object, the header in NiBabel is a dictionary.

[44]:
```
print(header1)
```

```
<class 'nibabel.nifti1.Nifti1Header'> object, endian='<'
sizeof_hdr      : 348
data_type       : b''
db_name         : b''
extents         : 0
session_error   : 0
regular         : b'r'
dim_info        : 54
dim             : [  3 176 256 256   1   1   1   1]
intent_p1       : 0.0
intent_p2       : 0.0
intent_p3       : 0.0
intent_code     : none
datatype        : int16
bitpix          : 16
slice_start     : 0
pixdim          : [1.   1.   1.   1.   1.95 0.   0.   0.  ]
vox_offset      : 0.0
```

```
scl_slope       : nan
scl_inter       : nan
slice_end       : 0
slice_code      : unknown
xyzt_units      : 10
cal_max         : 0.0
cal_min         : 0.0
slice_duration  : 0.0
toffset         : 0.0
glmax           : 0
glmin           : 0
descrip         : b'TE=2.3;Time=111930.158;phase=1'
aux_file        : b''
qform_code      : scanner
sform_code      : scanner
quatern_b       : -0.12108317762613297
quatern_c       : 0.027722394093871117
quatern_d       : -0.04943510890007019
qoffset_x       : -107.91807556152344
qoffset_y       : -117.78742980957031
qoffset_z       : -58.31475830078125
srow_x          : [ 9.93575513e-01  9.12692249e-02  6.69185743e-02
-1.07918076e+02]
srow_y          : [-1.0469611e-01  9.6579009e-01  2.3725149e-01 -1.1778743e+02]
srow_z          : [-4.2975545e-02 -2.4273333e-01  9.6914065e-01 -5.8314758e+01]
intent_name     : b''
magic           : b'n+1'
```

2. The **nifti affine** is a 4x4 matrix that encodes the spatial interpretation of the data. Each voxel (i, j, k) in the data represents the MR signal present in a given point in space (x, y, z); thus, the affine is a transformation between voxel space and scanner space. Be sure to check this very good documentation. At the end of section 3 we will discuss a bit more the concept of the image affine.

[45]:
```
print(affine1)
```

```
[[ 9.93575513e-01  9.12692249e-02  6.69185743e-02 -1.07918076e+02]
 [-1.04696110e-01  9.65790093e-01  2.37251490e-01 -1.17787430e+02]
 [-4.29755449e-02 -2.42733330e-01  9.69140649e-01 -5.83147583e+01]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.00000000e+00]]
```

3. The **data** is numpy array where each entry is the value of the reconstructed MR signal. This data however cannot be naively loaded and interpreted without any sort of spatial information.

[46]:
```
print(f"Type: {type(data1)}")
print(f"Shape: {data1.shape}")
print(f"Data:\n {data1}")
```

```
Type: <class 'numpy.ndarray'>
Shape: (176, 256, 256)
```

```
Data:
 [[[ 8.  4.  2. …  1.  1.  0.]
  [ 6.  4.  6. …  4.  2.  0.]
  [ 5.  8.  9. …  0.  3.  0.]
  …
  [ 6.  9.  1. …  3.  2.  0.]
  [ 3.  5.  9. …  4.  2.  0.]
  [ 0.  0.  0. …  0.  0.  0.]]

 [[ 5.  3. 10. …  3.  2.  0.]
  [ 9.  1.  4. …  0.  2.  0.]
  [ 1.  6.  6. …  3.  5.  0.]
  …
  [ 7.  2.  0. …  1.  1.  0.]
  [ 5.  7.  4. …  4.  0.  0.]
  [ 0.  0.  0. …  0.  0.  0.]]

 [[ 7.  7.  9. …  2.  1.  0.]
  [ 0.  1. 10. …  2.  1.  0.]
  [ 5.  4.  6. …  3.  1.  0.]
  …
  [14.  4. 14. …  0.  3.  0.]
  [ 7.  1.  5. …  4.  3.  0.]
  [ 0.  0.  0. …  0.  0.  0.]]

 …

 [[13. 16.  5. …  4.  4.  0.]
  [21. 12. 19. …  2.  5.  0.]
  [ 3.  4. 11. …  3.  4.  0.]
  …
  [ 7.  0. 10. …  4.  3.  0.]
  [14.  8. 11. …  2.  2.  0.]
  [ 0.  0.  0. …  0.  0.  0.]]

 [[ 1.  3.  5. …  8.  5.  0.]
  [ 3. 10.  4. … 10.  5.  0.]
  [12.  9. 17. …  6.  3.  0.]
  …
  [ 1.  7. 11. …  0.  2.  0.]
  [ 6. 13.  5. …  1.  1.  0.]
  [ 0.  0.  0. …  0.  0.  0.]]

 [[10.  6.  4. …  2.  2.  0.]
  [ 3.  6. 16. …  2.  1.  0.]
  [12.  0. 16. …  4.  4.  0.]
  …
  [16.  6. 13. …  1.  6.  0.]
```

```
[21. 12.  7. … 7.  3.  0.]
[ 0.  0.  0. … 0.  0.  0.]]]
```

Of course, if you overwrite any of these three attributes you can save a new file with the new data. You can do that by simply typing: `nib.save(nib.Nifti1Header(new_data, new_affine, new_header), new_file_name)`. However, overwriting any of these things without clear knowledge of what you are doing (specially for the header and the affine) could have disastrous consequences. Therefore, it is good practice to use dedicated neuroimaging softwares for certain operations with the data.

1.2 2. Data visualization: What do I have? How do I see what I have?

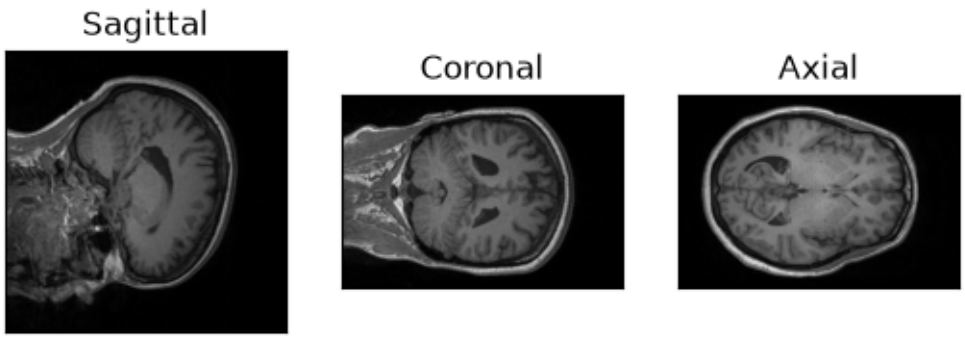
As you may have realized, neuroimaging data is complex. Each image has a lot of information that allows any person and/or software to interpret the data correctly. Besides being out of scope, there is not enough time in 1.5h to cover all the technical details around the NIFTI images. Yet, we are now in a position to start taking a proper look into them and, hopefully, provide a comprehensive overview.

Let's start by plotting one scan in the simplest form possible (e.g., the one we have already loaded in memory). Have in mind that each image is a 3D **volume** so we will have to plot slices.

[47]:
```
fig, ax = plt.subplots(1,3)
ax[0].imshow(data1[70,...], cmap='gray')
ax[0].set_xticks([]), ax[0].set_yticks([]), ax[0].set_title("Sagittal")

ax[1].imshow(data1[:,90,:], cmap='gray')
ax[1].set_xticks([]), ax[1].set_yticks([]), ax[1].set_title("Coronal")

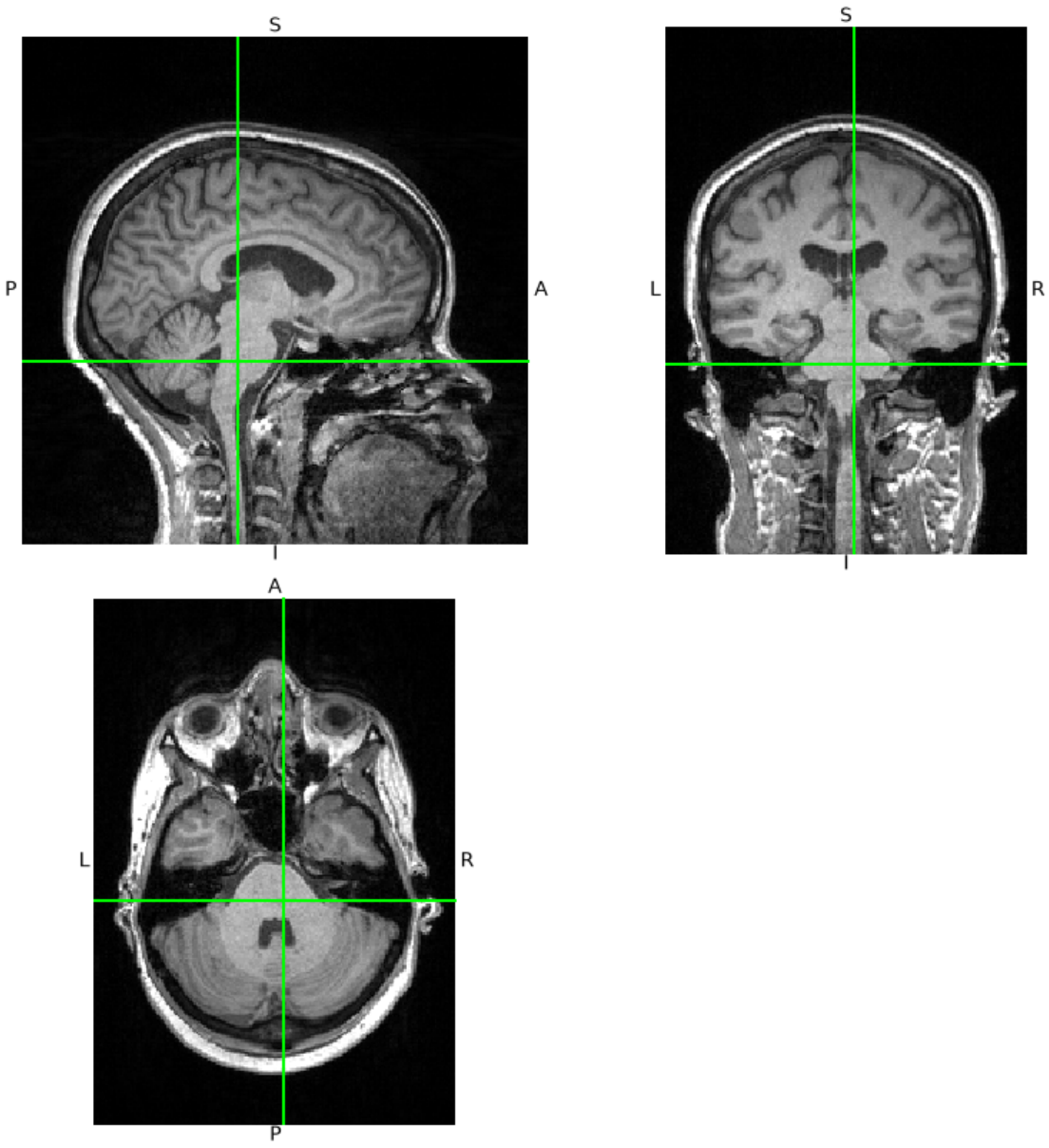
ax[2].imshow(data1[...,125], cmap='gray')
ax[2].set_xticks([]), ax[2].set_yticks([]), ax[2].set_title("Axial")
plt.show()
```



A faster option is provided by the attributes of the nibabel oject but it does not allow control over which planes to show.

[48]:
```
img1.orthoview()
```

[48]: <OrthoSlicer3D: BIDS-converted/sub-FCS01ACM/anat/sub-FCS01ACM_acq-series4_T1w.nii.gz (176, 256, 256)>



The first advantage of matplotlib is the control that we have over which plane we wish to show. We can rotate the numpy array, transpose it, select which planes to show, … something we were not able to do with a DICOM series (i.e., a list of 2D slices). The obvious advantage of using the `orthoview()` method is that it is very fast, so it would be useful for quickly inspecting the images. Yet, we can do better with Nilearn. More specifically, if your intention is to simply visualize a nifti image the view_image function will output an interactive plot of your scan. The function offers many versatile arguments which may be worth being familiar with.

[49]:
```
!pip install nilearn
from nilearn import plotting
```

```
Collecting nilearn
  Downloading nilearn-0.14.1-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: jinja2>=3.1.6 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nilearn) (3.1.6)
Collecting joblib>=1.2.0 (from nilearn)
  Downloading joblib-1.6.0-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: nibabel>=5.2.0 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nilearn) (5.4.2)
Requirement already satisfied: numpy>=1.26.0 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nilearn) (2.4.6)
Requirement already satisfied: packaging>=26.0.0 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nilearn) (26.3)
Collecting pandas>=2.3.0 (from nilearn)
  Downloading pandas-3.0.6-cp311-cp311-macosx_11_0_arm64.whl.metadata (79 kB)
Requirement already satisfied: requests>=2.33.0 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nilearn)
(2.34.2)
Collecting scikit-learn!=1.7.0,!=1.9.0,>=1.5.0 (from nilearn)
  Downloading scikit_learn-1.9.1-cp311-cp311-macosx_12_0_arm64.whl.metadata (9.3
kB)
Requirement already satisfied: scipy>=1.11.1 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from nilearn)
(1.17.1)
Requirement already satisfied: MarkupSafe>=2.0 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
jinja2>=3.1.6->nilearn) (3.0.4)
Collecting cloudpickle>=3.0 (from joblib>=1.2.0->nilearn)
  Downloading cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)
Requirement already satisfied: importlib-resources>=5.12 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
nibabel>=5.2.0->nilearn) (7.1.0)
Requirement already satisfied: typing-extensions>=4.6 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
nibabel>=5.2.0->nilearn) (4.16.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
pandas>=2.3.0->nilearn) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from python-
dateutil>=2.8.2->pandas>=2.3.0->nilearn) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
requests>=2.33.0->nilearn) (3.5.1)
Requirement already satisfied: idna<4,>=2.5 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
```

```
requests>=2.33.0->nilearn) (3.20)
Requirement already satisfied: urllib3<3,>=1.26 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
requests>=2.33.0->nilearn) (2.8.0)
Requirement already satisfied: certifi>=2023.5.7 in
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-packages (from
requests>=2.33.0->nilearn) (2026.7.22)
Collecting narwhals>=2.0.1 (from scikit-learn!=1.7.0,!=1.9.0,>=1.5.0->nilearn)
  Downloading narwhals-2.27.1-py3-none-any.whl.metadata (15 kB)
Collecting threadpoolctl>=3.5.0 (from scikit-
learn!=1.7.0,!=1.9.0,>=1.5.0->nilearn)
  Downloading threadpoolctl-3.7.0-py3-none-any.whl.metadata (24 kB)
Downloading nilearn-0.14.1-py3-none-any.whl (11.5 MB)
                         11.5/11.5 MB
11.2 MB/s  0:00:01m0:00:010:01
Downloading joblib-1.6.0-py3-none-any.whl (306 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
Downloading pandas-3.0.6-cp311-cp311-macosx_11_0_arm64.whl (10.0 MB)
                         10.0/10.0 MB
11.5 MB/s  0:00:00eta 0:00:01
Downloading scikit_learn-1.9.1-cp311-cp311-macosx_12_0_arm64.whl (8.3 MB)
                         8.3/8.3 MB
11.1 MB/s  0:00:00 eta 0:00:01
Downloading narwhals-2.27.1-py3-none-any.whl (483 kB)
Downloading threadpoolctl-3.7.0-py3-none-any.whl (26 kB)
Installing collected packages: threadpoolctl, narwhals, cloudpickle, pandas,
joblib, scikit-learn, nilearn
                         7/7
[nilearn]m6/7 [nilearn]earn]
Successfully installed cloudpickle-3.1.2 joblib-1.6.0 narwhals-2.27.1
nilearn-0.14.1 pandas-3.0.6 scikit-learn-1.9.1 threadpoolctl-3.7.0
```

[50]:
```
plotting.view_img(
    img1, bg_img=False, colorbar=False, cmap='gray', symmetric_cmap=False
)
```

```
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/ipykernel_72381/1117078080.py:1
: UserWarning: Resampling binary images with continuous or linear interpolation.
This might lead to unexpected results. You might consider using nearest
interpolation instead.
  plotting.view_img(
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/ipykernel_72381/1117078080.py:1
: UserWarning: Casting data from int32 to float32
  plotting.view_img(
```

[50]: `<nilearn.plotting.html_stat_map.StatMapView at 0x30d9d8090>`

Question: Why is nilearn using the NIFTI object instead of just the numpy array?

Answer: The numpy array is just voxel values with no spatial meaning. The NIfTI object also carries the affine (voxel → mm world coordinates) and the header (voxel size, orientation). Nilearn needs these to put the image in real space, orient the axes correctly, and resample and overlay images on different grids.

This way of plotting images offers the posisbility of overlaying two different data points (e.g., scans from two different subjects). In other type of experiments, where some statistical procedures would have allowed us to identify active parts of the brain in certain conditions, the same function could be used to overlay the brain and its activity. For now, let's simply overlay two images to check the correspondance between them.

[54]:
```
# Load data from a second subject
img2 = nib.load("./BIDS-converted/sub-FCS02AMC/anat/
 ↪sub-FCS02AMC_acq-series4_T1w.nii.gz")
header2, affine2, data2 = img2.header, img2.affine, img2.get_fdata()

plotting.view_img(
    img1, bg_img=img2, opacity=.4,
    colorbar=False, cmap='hot', symmetric_cmap=False
)
```

```
/opt/anaconda3/envs/ML4Neuro/lib/python3.11/site-
packages/numpy/_core/fromnumeric.py:840: UserWarning: Warning: 'partition' will
ignore the 'mask' of the MaskedArray.
  a.partition(kth, axis=axis, kind=kind, order=order)
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/ipykernel_72381/2980838870.py:5
: UserWarning: Casting data from int16 to float32
  plotting.view_img(
/var/folders/2h/f57zt3hn42x1qlhr9qgn_l2h0000gn/T/ipykernel_72381/2980838870.py:5
: UserWarning: Casting data from int32 to float32
  plotting.view_img(
```

[54]:
```
<nilearn.plotting.html_stat_map.StatMapView at 0x3096ec890>
```

As you can see, the two scanners do not overlay between themselves. You may ask the question, "why?" but the answer might not be trivial. A better question is the same one we have asked before: *Why is nilearn using the whole NIFTI object instead of the data to do the plotting?* Let's take a closer look at properties of the objects we have just plotted.

[60]:
```
print("Affine from sub-FCS01ACM")
print(affine1)
print("================================================")
print("Affine from sub-FCS02AMC")
print(affine2)
```

```
Affine from sub-FCS01ACM
[[ 9.93575513e-01  9.12692249e-02  6.69185743e-02 -1.07918076e+02]
 [-1.04696110e-01  9.65790093e-01  2.37251490e-01 -1.17787430e+02]
 [-4.29755449e-02 -2.42733330e-01  9.69140649e-01 -5.83147583e+01]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.00000000e+00]]
```

```
==========================================
Affine from sub-FCS02AMC
[[ 9.97337222e-01  5.32575697e-02 -4.97353673e-02 -8.79803696e+01]
 [-4.98367734e-02  9.96463418e-01  6.76527098e-02 -8.78155518e+01]
 [ 5.31622656e-02 -6.49941936e-02  9.96468544e-01 -1.06198227e+02]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.00000000e+00]]
```

What was the image affine codifying? Exactly! The spatial coordinates of each voxel. Let us discover the spatial position of voxel $(60, 140, 140)$ for each one of the scans. To do so, we simply multiply the affine with the coordinates in voxel space:

[56]:
```
voxel = np.array([60,140,140,1]) # Number 1 is for matrix multiplication
xyz_1 = affine1 @ voxel
xyz_2 = affine2 @ voxel
print("Cartesian coordinates in mm for image 1: ", xyz_1[:-1])
print("Cartesian coordinates in mm for image 2: ", xyz_2[:-1])
```

```
Cartesian coordinates in mm for image 1:  [-26.15725288  44.35662526
40.80373371]
Cartesian coordinates in mm for image 2:  [-27.64702791  58.17049973
27.39791806]
```

It is not surprising that the overlay between the two subjects was not good. Each scan was in its own space (i.e., subject space). Comparing subjects and images is not as straightforward as plotting them together!

All these plots and inspections can be done much easier and (probably) faster with neuroimaging softwares like FSL, MRtrix, ANTs or 3Dslicer. When it comes to plain visualization of nifti objects, it is recommended to have at least one of these three available. In fact, FSL or MRtrix incorporate a lot of useful tools for working with neuroimaging data including but not limited to co-registration and segmentation algorithms. Some of them have overlapping methods while some other have unique software capabilities. Except for 3D slicer, LINUX is the recommended operating system.

1.3 3. Image registration and tissue segmentation: Unsupervised vs supervised methods

FSL fast and flirt? ANTs? These are softwares that might be cumbersome to install for everybody, but nonetheless, they should be explained. And probably a tour guide through the most important FSL features will come in handy.

Data (pre)processing for neuroimaging includes spatial transformation steps.: - removal of the non-brain matter (i.e., skull stripping) - alignment for images that are located in different grids to ensure that the same spatial coordinates denote the same brain region in different patients - segmenting different types of brain tissue (white matter, gray matter and cerebrospinal fluid) - to be able to restrain certain analyses for specific tissues - a denoising technique known as removal of bias field homogeneities

Such steps are typically performed with specialized neuroimaging software, that provides resonably accurate results for 3d and 4d images. Most popular neuroimaging software for running spatial transformations includes FSL, AFNI and ANTs.

In the following cell, we'll see several examples showcasing FSL's functionality, including: 1. bet (Brain Extraction Tool), a commonly used program to perform brain extraction, which is a type of segmentation used to separate brain (gray matter, white matter and CSF) and non-brain matter (skull, neck) 2. fast (FMRIB's Automated Segmentation Tool), which is a popular implementation for segmenting different brain tissues, usually in three classes: gray matter, white matter and CSF; this procedure is typically performed after non-brain matter was removed from the image (after bet) 3. flirt (FMRIB's Linear Registration Tool), which can run rigid body and affine registration between two images; in this example, we'll perform the registration of an atamocal scan to a standard space. Standard space is a population brain (i.e., and "average" brain) that provides a common reference and coordinate space to be able to combine data from different participants within the same study, or visualize/analyze results across studies.

For each dataset, depending on the quality of the data, you might need to fine tune some parameters for each step. However, as a benchmark, we will use values that have proven to be robust (based on our judgement) and will comment on other options if they fail.

[59]:
```
%%bash

export FSLDIR="$HOME/fsl"
export PATH="$FSLDIR/share/fsl/bin:$PATH"
source "$FSLDIR/etc/fslconf/fsl.sh"

# Directory and file names
Postprocessing_dir="./FSL-results/sub-FCS01ACM/anat/"
nifti="./BIDS-converted/sub-FCS01ACM/anat/sub-FCS01ACM_acq-series4_T1w.nii.gz"

# Create derivative directory
mkdir -p "$Postprocessing_dir"

# 1) Display header information
#fslhd $nifti

# 2) Skull-stripping or brain extraction
masked_brain="${Postprocessing_dir}sub-FCS01ACM_T1w_brain"
bet "$nifti" "${masked_brain}.nii.gz"    #-f .5 -g -.6
#bet $masked_brain.nii.gz $masked_brain.nii.gz -f .35 -m


# 3) Tissue segmentation
#fast -n 4 -I 25 -W 25 -H .25 $masked_brain.nii.gz
#fast $masked_brain.nii.gz

# 4) Registration to template
reference="$FSLDIR/data/standard/MNI152_T1_1mm_brain.nii.gz"
registered="${Postprocessing_dir}sub-FCS01ACM_T1w_brain_space-MNI152"
transformation="${Postprocessing_dir}sub-FCS01ACM_matrix2MNI152.txt"
flirt -in "${masked_brain}.nii.gz" -ref "$reference" -out "${registered}.nii.
 ↪gz" -omat "$transformation"
```

```
#flirt -in $masked_brain.nii.gz -ref $reference -out $registered -applyxfm␣
↪-init $transformation
```

To inspect the outputs of each step you can either do it like in this notebook or by typing `fsleyes` into you terminal. Once the viewer has opened, drag the images you want to inspect in the black screen. As you will see, this second option is much more convenient!

All of of the previous methods are unsupervised; i.e., they attemp to minimize a cost function defined on properties of the data. Here are a few examples:

- In the case of `fast`, mixture gaussian models and hidden markov radiant fields (HMRF) are used. Mixture models allow to define a threshold based on the intensity of the pixels. Added to it, HMRF exploit the fact that two neightbouring models are likely to be of the same type. The `fast` command automatically corrects for bias field homogeneities (i.e., the image might look brighter in posterior regions of the brain. Don't worry there are plenty of resources where you can find this and more detailed information: FAST (or go to the original references!)
- In the case of `flirt` the cost function can be the correlation or the mutual information between the two images (amongst others).
- As in many other examples in machine learning, despite being rather obvious for a human, the `bet` algorithm is unintuitively complex. It relies on histogram thresholding and iterative surface estimating.

The only way to become profficient with these steps and algorithms is to fight with them and with some data. Feel free to do so! However, for some data it might be difficult. ANTs might provide more accurate solutions but their tools are substantially heavier in terms of computation. Deep learning might also be a valuable tool. Yet, deep learning usually requires supervised methods with ground-truth labels. Potentially problematic is the "generalizability" of the training and validation dataset. Other possibilities that might improve the final results is the usage of multimodal data.